

07_freshentry_walkforward

June 30, 2026

1 07 — Fresh-entry walk-forward (the fair final test)

Notebook 06 entered on a *quarterly snapshot* — so for a position a sharp opened weeks earlier, it bought late, after the move. That’s unfair to the actual strategy, which would copy sharps *when they enter*. This notebook fixes that and is the honest final check.

What it does: for every bet a sharp ever made, it asks — *was that sharp already skilled, judged only on trades resolved before this entry?* (point-in-time qualification). If yes, the entry is a real-time signal. We then “buy” at **the market price on the day they entered** (CLOB price-history), hold to resolution, and measure the outcome — out-of-sample, by how many qualified sharps entered the same side.

Guardrail against the last mirage: we report **edge** (hit-rate — price), and BOTH the **mean and median** return. Last time a couple of 10¢ longshots faked a positive average; the median and the edge will keep us honest.

Residual caveat (unchanged): the candidate pool is today’s leaderboard, so a little survivorship remains — but qualification and outcomes are point-in-time.

```
[1]: import importlib, pmc
importlib.reload(pmc)
from pmc import CFG, get_leaderboard, get_closed_positions, price_at
import pandas as pd, numpy as np, time
from datetime import datetime, timezone

NOW = int(time.time())
print(f"trailing qualification gate: >= {CFG.WF_MIN_TRAILING_TRADES} resolved_
↳bets "
      f"AND win-rate >= {CFG.WF_MIN_TRAILING_WINRATE} | realistic price = {CFG.
↳WF_USE_PRICE_HISTORY}")
```

```
trailing qualification gate: >= 30 resolved bets AND win-rate >= 0.55 |
realistic price = True
```

1.1 1. Candidate pool + full resolved history

Same data as notebook 06 (cached, so this is fast on re-run): every candidate’s resolved bets with entry time, resolution time, and outcome.

```
[2]: cands = {}
for w in ("ALL", "MONTH"):
    for r in get_leaderboard(window=w, limit=CFG.WF_CANDIDATES):
        cands.setdefault(r["wallet"], r)

rows = []
for i, wallet in enumerate(cands):
    for p in get_closed_positions(wallet, max_positions=600):
        rows.append({
            "wallet": wallet, "asset": p.get("asset"),
            "conditionId": p.get("conditionId"), "outcome": p.get("outcome"),
            "entry": float(p.get("avgPrice") or 0), "entry_ts": int(p.
↳get("timestamp") or 0),
            "endDate": p.get("endDate"),
            "won": 1 if float(p.get("realizedPnl") or 0) > 0 else 0,
        })
    if (i + 1) % 50 == 0:
        print(f"  pulled {i+1}/{len(cands)}")

h = pd.DataFrame(rows)
h["res_ts"] = (pd.to_datetime(h["endDate"], errors="coerce", utc=True).
↳astype("int64") // 10**9)
h = h[(h["entry_ts"] > 0) & (h["res_ts"] > 0) & (h["res_ts"] > h["entry_ts"]) &
↳(h["entry"] > 0)].copy()
print(f"{len(h)} usable resolved bets | {h['wallet'].nunique()} wallets")
```

```
pulled 50/248
pulled 100/248
pulled 150/248
pulled 200/248
20777 usable resolved bets | 208 wallets
```

1.2 2. Point-in-time qualification of every entry

For each entry event we count that wallet’s own bets that had **resolved before** the entry, and its trailing win-rate — using `searchsorted`, so it’s fast. The entry is a “qualified signal” only if the wallet already cleared the trailing gate at that moment.

```
[3]: h = h.sort_values("entry_ts").reset_index(drop=True)
qualified_mask = np.zeros(len(h), dtype=bool)

for wallet, idx in h.groupby("wallet").groups.items():
    w = h.loc[idx]
    res_sorted = w.sort_values("res_ts")
    res_times = res_sorted["res_ts"].values
    win_cumsum = np.cumsum(res_sorted["won"].values)
    # for each of this wallet's entries, how many of its bets had resolved
↳before entry_ts
```

```

k = np.searchsorted(res_times, w["entry_ts"].values, side="left")
n_prior = k
wins_prior = np.where(k > 0, win_cumsum[np.clip(k - 1, 0, len(win_cumsum) -
↳1)], 0)
wr_prior = np.where(n_prior > 0, wins_prior / np.maximum(n_prior, 1), 0.0)
ok = (n_prior >= CFG.WF_MIN_TRAILING_TRADES) & (wr_prior >= CFG.
↳WF_MIN_TRAILING_WINRATE)
qualified_mask[np.array(idx)] = ok

q = h[qualified_mask].copy()
print(f"{len(q)} point-in-time-qualified entry events "
      f"({len(q)/max(len(h),1)*100:.1f}% of all entries), {q['wallet'].
↳nunique()} distinct sharps")

```

5229 point-in-time-qualified entry events (25.2% of all entries), 54 distinct sharps

1.3 3. Build trades: enter at the Nth qualified sharp's entry, at that day's price

Per market-side, order the distinct qualified sharps by when they first entered. For threshold N, the signal triggers at the **Nth** sharp's entry — we buy at the market price *that day* and hold to resolution.

```

[4]: def build_trades(n):
    out = []
    for (cid, outcome), g in q.groupby(["conditionId", "outcome"]):
        firsts = g.groupby("wallet")["entry_ts"].min().sort_values()
        if len(firsts) < n:
            continue
        enter_ts = int(firsts.iloc[n - 1])           # time the Nth sharp
↳had entered
        asset = g["asset"].iloc[0]
        won = int(g["won"].iloc[0])
        # realistic price the day the signal completed; fallback to that
↳sharp's own entry
        buy = price_at(asset, enter_ts) if CFG.WF_USE_PRICE_HISTORY else None
        if buy is None:
            nth_wallet = firsts.index[n - 1]
            buy = float(g[g["wallet"] == nth_wallet]["entry"].iloc[0])
        buy = min(float(buy) + CFG.SLIPPAGE_TOLERANCE, 0.99)
        if not (CFG.PRICE_FLOOR <= buy <= CFG.PRICE_CEILING):
            continue
        out.append({"conditionId": cid, "outcome": outcome, "backers":
↳int(len(firsts)),
                    "buy": round(buy, 3), "won": won,
                    "ret": round(((1 - buy) / buy) if won else -1.0, 3)})
    return pd.DataFrame(out)

```

```
trades_by_n = {n: build_trades(n) for n in [1, 2, 3]}
for n, t in trades_by_n.items():
    print(f"backers>={n}: {len(t)} trades")
```

```
backers>=1: 1400 trades
backers>=2: 228 trades
backers>=3: 92 trades
```

1.4 4. The verdict

edge = hit-rate – avg price (positive = real edge). mean_ret is skewed by longshots; median_ret and pct_profitable are the robust reality check. If edge = 0 and median_ret < 0, there is no tradeable signal.

```
[5]: def summarize(t, n):
    if t is None or t.empty:
        return None
    return {
        "min_backers": n, "n_bets": len(t),
        "hit_rate": round(t["won"].mean(), 3),
        "avg_price": round(t["buy"].mean(), 3),
        "edge": round(t["won"].mean() - t["buy"].mean(), 3),
        "mean_ret": round(t["ret"].mean(), 3),
        "median_ret": round(t["ret"].median(), 3),
        "pct_profitable": round((t["ret"] > 0).mean(), 3),
    }

res = pd.DataFrame([r for r in (summarize(trades_by_n[n], n) for n in [1, 2, 3]) if r])
print("FRESH-ENTRY WALK-FORWARD - out-of-sample, entered at the sharps' own entry-day price:")
res
```

FRESH-ENTRY WALK-FORWARD - out-of-sample, entered at the sharps' own entry-day price:

```
[5]:
```

	min_backers	n_bets	hit_rate	avg_price	edge	mean_ret	median_ret	\
0	1	1400	0.450	0.487	-0.037	0.413	-1.0	
1	2	228	0.434	0.477	-0.043	0.372	-1.0	
2	3	92	0.348	0.462	-0.114	0.271	-1.0	

	pct_profitable
0	0.450
1	0.434
2	0.348

1.5 What the answer means

- **edge > 0, median_ret > 0, on a healthy n_bets** → there is a real, point-in-time edge in copying fresh sharp entries. This would be the version worth paper-trading. Add theme clustering / a wider active roster to lift frequency, then paper-trade 4–6 weeks before any capital.
- **edge = 0 and median_ret = 0** → even copying sharps *the moment they enter* doesn't beat the price. Combined with notebook 06, that's a clean, evidence-based **stop**: the public smart-money edge on Polymarket is arbitrated away. No shame in that — it's the efficient-markets result, and you found it for the price of compute instead of capital.

Whatever it shows, this is the experiment that settles it. Send me the table and I'll give you the honest read and, if it's a stop, a one-page written summary of everything we learned.